

MAN Energy Solutions

Product domain:	Electronics & Software		
Subject:			
Title:	CAN-bus Controller Specification		
Number:		No. of pages:	20
Version:	0.2	Authors:	Mads Richardt
Date:	Date in 2025-12-16	Resp initials:	
Document state:	See document history	Keywords:	[TRIT-3880]
Abstract			

Copyright © MAN Energy Solutions

This document, together with its supplements, is the property of MAN Energy Solutions and subject to copyright protection. The material is placed at the disposal of the recipient solely for his own use, and only for the manufacture, repair or use of original MAN Energy Solutions products, according to contract or written agreement herein with MAN Energy Solutions. The material must not, either wholly or partly, be copied, reproduced, made public, or in any other way made available to any third party without the written consent to this effect from MAN Energy Solutions. The material must not be transferred or handed over to any third party, neither direct nor by succession to the rights of the recipient.

Document history		
Version label Dated	Authors	Changes
0.1 2025-03-18	TMYAES	Initial document.
0.2 2025-12-16	AFNI	Updates to the states.

Table of Contents

Table of Contents	3
1. About this document	4
1.1 Purpose and Scope.....	4
1.2 Intended Audience and Reading Guidance	4
1.3 Abbreviations	5
1.4 References.....	6
2. Introduction	7
2.1 CAN Key Concepts	7
2.1.1 Frame transmissions	7
2.1.2 Bus access method	7
2.1.3 Remote data request	8
2.1.4 Node States.....	8
2.1.4.1 Error Active	8
2.1.4.2 Error passive.....	8
2.1.4.3 Bus-Off.....	8
2.1.5 Bit Timing and Synchronization.....	8
2.1.6 Error and overload detection and handling.....	9
2.1.6.1 Bit error	9
2.1.6.2 Stuff error.....	9
2.1.6.3 CRC error.....	9
2.1.6.4 Form error	9
2.1.6.5 ACK error.....	10
2.1.6.6 Overload	10
2.1.7 The LLC Frame Structure	10
2.1.8 The MAC Frame Structure.....	10
3. Requirements	13
4. CAN_Ctrl Module Design	14
4.1.1 CAN_fsm	16
4.1.2 Serial_to_Aavalon-ST (STA).....	18
4.1.3 Avalon-ST_to_Serial (ATS).....	19
4.1.4 Node Clock (NC).....	20
4.1.5 CRC module	20

1. About this document

1.1 Purpose and Scope

This document specifies a Controller Area Network (CAN) control module (CAN_Ctrl), see [\[TRIT-3880\]](#).

This CAN-bus controller is meant to be used in the first IO-extender, to communicate with a NOx-sensor, as shown in Figure 1. Input to the controller will be a CAN-bus frame controlled by SW running on the CPU_1400. Output will be all correctly received frames from the NOx-sensors.

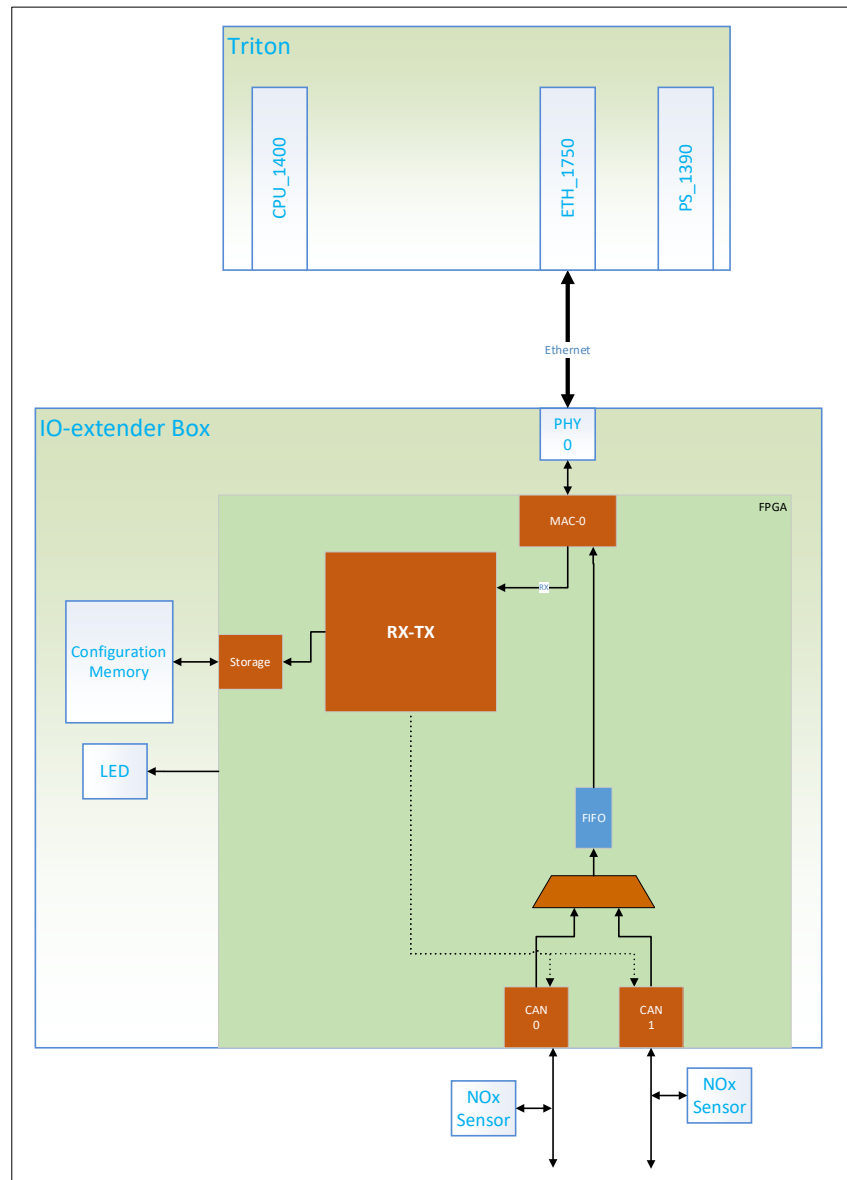


Figure 1. IO-extender overview.

1.2 Intended Audience and Reading Guidance

The specification is intended for software and hardware developers that uses the module.

1.3 Abbreviations

Abbreviation	Meaning	Description
ACK	Acknowledgement	
AEF	Active Error Frame	Six consecutive DBs
BT	Bit Time	The duration of one CAN bit, composed of Sync_Seg, Prop_Seg, Phase_Seg1, and Phase_Seg2. Defined as $BT = TQ * (Sync_Seg + Prop_Seg + Phase_Seg1 + Phase_Seg2)$.
BO	Bus-Off	see section 2.1.4.3
BRP	Baud Rate Pre-scaler	Configurable pre-scaler that multiplies the system clock period to generate the TQ.
BTC	BT Counter	A counter that tracks the BT by counting TQ cycles through the CAN bit segments (Sync_Seg, Prop_Seg, Phase_Seg1, Phase_Seg2).
CNC	CAN_Ctrl Node Clock	see section 4.1.4
CAN	Controller Area Network	A multi-node serial communication protocol used in automotive and industrial applications.
CRC	Cyclic Redundancy Check	A 15-bit error-detection code ensuring frame integrity.
DB	Dominant Bit	A dominant signal on the bus is represented by a logic '0'.
DLC	Data Length Code	Specifies the number of data bytes (0-8) in the payload.
DF	Data Frame	A frame that carries data from sender to receiver.
EA	Error Active	see section 2.1.4.1
EF	Error Frame	A frame sent to signal a detected error on the bus.
EFD	Error Frame Delimiter	Eight consecutive RBs, marking the end of an error frame.
EP	Error Passive	see section 2.1.4.2
EOF	End Of Frame	Seven consecutive RBs, marking the end of a valid frame.
ID	Frame Identifier	The unique 11-bit (CAN 2.0A) or 29-bit (CAN 2.0B) identifier used for arbitration and message priority. Lower ID values have higher priority on the bus.
IDE	Identifier Extension Bit	DB = Standard ID (11-bit), RB = Extended ID (29-bit).
IFS	Inter-frame Space	The IFS consist of the INT, ST and bus idle fields.
INT	Intermission	Three consecutive RBs.
LL	Link Layer	The OSI layer responsible for frame transmission and reception.
LLC	Logical Link Control	The higher sub-layer of the LL.
MAC	Medium Access Control	The lower sub-layer of the LL.
NS	Node State	A 2-bit signal indication the CAN_Ctrl node state. '00' = EA, '01' = AP or '11' = BO. '10' is not used.
OF	Overload Frame	Six consecutive DBs.
OFD	Overload Frame Delimiter	Eight consecutive RBs.

Abbreviation	Meaning	Description
PEF	Passive Error Frame	Six consecutive RBs.
RB	Recessive bit	A recessive signal on the bus is represented by a logic '1'.
RF	Remote Frame	A frame requesting the transmission of a corresponding DF.
REC	Receive error count	Tracks receive errors. This value gets incremented by a receiving node when a reception fails and decremented when a reception succeeds.
RTR	Remote Transmission Request	'0' = DF, '1' = RF.
SJW	Synchronization Jump Width	The maximum allowed phase correction (in TQ) during resynchronization.
SOF	Start Of Frame	A DB marking the start of a new frame.
SRR	Substitute Remote Request	Used in CAN 2.0B frames, must be RB.
ST	Suspend Transmission	An EP node that transmitted the previous frame must wait eight BTs after INT before starting a new transmission. If another node transmits during this time, the suspended node switches to receiver mode.
TEC	Transmit error count	Tracks transmit errors. This value gets incremented by transmitting nodes when a transmission fails and decremented when a transmission succeeds.
TQ	Time Quantum	The smallest unit of CAN bit timing, determined by BRP and the system clock period (T_sysclk) as $TQ = BRP * T_{sysclk}$.

1.4 References

Reference	Author
1. Title: Road vehicles – Controller area network (CAN) – Part 1: Data link layer and physical signaling. Standard number: ISO 11898-1:2024.	ISO

2. Introduction

The CAN_Ctrl covers the CAN Link Layer (LL) according to ISO 11898-1 for CAN 2.0 A and B and is responsible for frame transmission, reception, arbitration, and error detection in a multi-node CAN network. Logical Link Control (LLC) frames are passed to/from a CAN_Ctrl node through Avalon-ST interfaces, and additional control signals indicate node status (see section 2.1.4) and frame transmission acknowledgment.

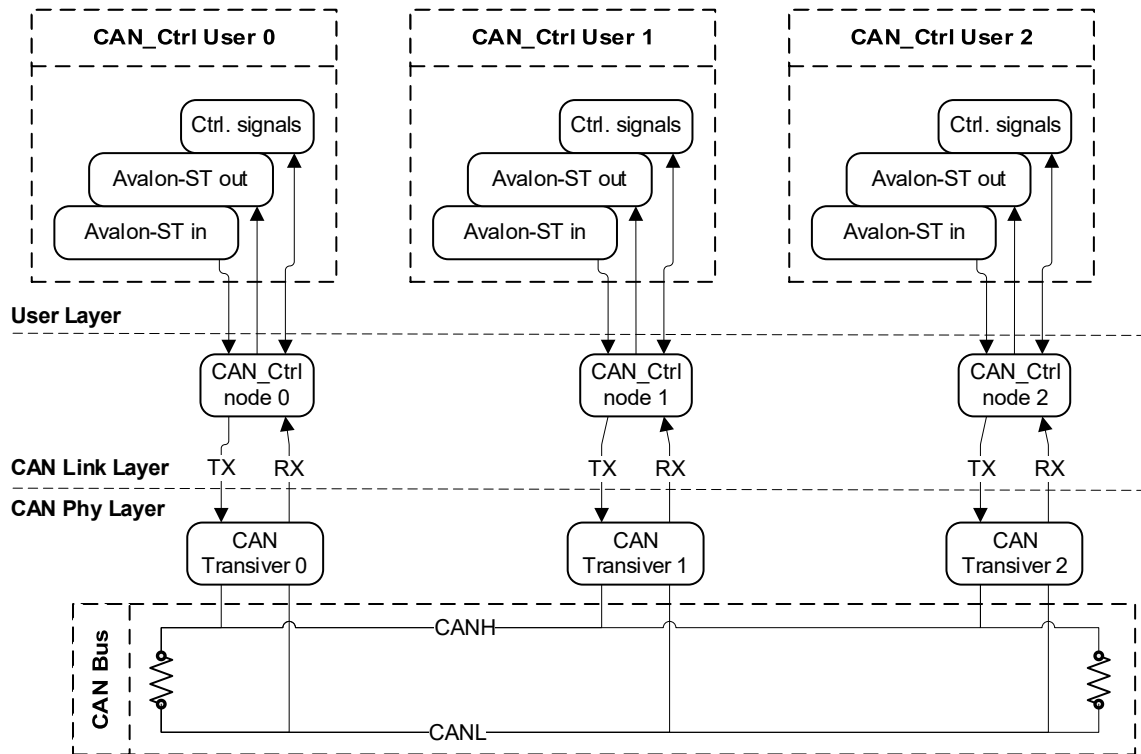


Figure 2. An example CAN containing three CAN_Ctrl nodes and three CAN_Ctrl node users. The CAN_Ctrl nodes constitute the CAN LL, and the CAN Bus and CAN transceivers constitute the CAN Phy Layer.

2.1 CAN Key Concepts

2.1.1 Frame transmissions

The CAN LL transmits four types of frames: Data Frames (DF), Remote Frames (RF), Error Frames (EF) and Overload Frames (OF). When the bus is idle a CAN node can initiate the transmission of a DF or RF. If an error condition occurs (bit error, stuff error, CRC error, form error or ack error), the node transmits an EF and If an overload condition is detected it transmits and OF, see section 2.1.6.

2.1.2 Bus access method

When multiple nodes attempt to transmit DFs or RFs simultaneously, arbitration determines bus access based on the Frame Identifier (ID). This process ensures that no data or time is lost during contention. The node with the highest priority ID continues transmission, while lower-priority nodes cease sending. If a DF and an RF share the same identifier, the DF prevails in arbitration and is transmitted.

2.1.3 Remote data request

By transmitting an RF, a node requests data from another node, prompting it to send the corresponding DF. The RF and its corresponding DF share the same identifier and use the same Data Length Code (DLC).

2.1.4 Node States

A CAN node operates in one of three bus states depending on its error condition: Error Active (EA), Error Passive (EP), or Bus-Off (BO). These states determine how the node participates in network communication and manages error signaling.

2.1.4.1 Error Active

A node starts in the EA state and fully participates in CAN communication. It sends an Active Error Frame (AEF) when it detects an error. A node remains EA if its Transmit Error Count (TEC) and Receive Error Count (REC) are both below 128.

2.1.4.2 Error passive

A node enters the EP state when either the TEC or REC exceeds 127. It sends a Passive Error Frame (PEF) when it detects an error. If the TEC drops below 128, the node returns to the EA state.

2.1.4.3 Bus-Off

A node enters the BO state when its TEC exceeds 255. In this state, the node does not send or receive frames and never transmits Dominant Bits (DB, logical '0'). Upon a restart request, the node must observe 128 occurrences of the bus being idle before it can attempt re-integration. Once the recovery conditions are met, the node resets its error counters to zero and re-enters the EA state.

2.1.5 Bit Timing and Synchronization

Each CAN bit is divided into four segments: Sync, Prop_Seg, Phase_Seg1, and Phase_Seg2, see Figure 3 and Table 1. The total Bit Time (BT) is measured in Time Quanta (TQ) and configured using the Baud Rate Pre-scaler (BRP). Synchronization ensures all nodes remain aligned with the bus. Hard synchronization occurs at the Start of Frame (SOF), resetting the Bit Timing Counter (BTC). Resynchronization adjusts timing on every subsequent RB-to-DB transition, correcting phase errors within the limits set by the Synchronization Jump Width (SJW). Bit stuffing guarantees regular transitions by inserting a stuff bit after five consecutive identical bits.

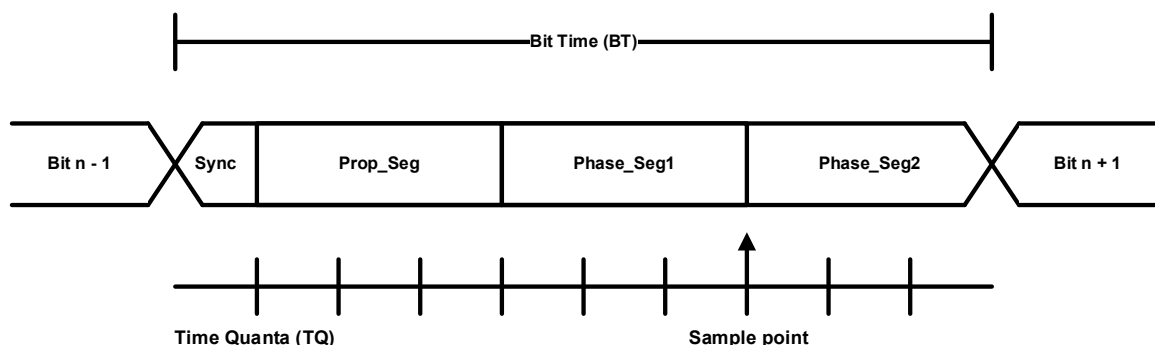


Figure 3. Phase segments of a CAN bit. Synchronization happens during the Sync segment and the bit is sampled at the end of Phase_Seg1.

Table 1. Bit timing parameter ranges defined by [1].

Parameter	Range
BRP	Integer from 1 to 32
Sync	1 TQ (fixed)
Prop_Seg	1 to 8 TQ
Phase_Seg1	1 to 8 TQ
Phase_Seg2	2 to 8 TQ (must be $\geq$ SJW)
SJW	1 to 4 TQ ($\leq$ Phase_Seg2)
Total	8 to 25 TQ

2.1.6 Error and overload detection and handling

CAN provides the following error detection mechanisms:

- **Bit Monitoring:** Transmitters compare the sent bit with the observed bit on the bus.
- **15-bit CRC (Cyclic Redundancy Check):** Both transmitters and receivers compute the CRC based on the actual bits observed on the bus.
- **Dynamic Bit Stuffing:** A stuff bit is inserted after five consecutive identical bits. This mechanism is used from the Start of Frame (SOF) up to and including the CRC field in the Medium Access Control (MAC) frame.
- **Frame Format Check:** Verifies compliance with the defined CAN MAC frame structure.
- **Acknowledgement (ACK) Check:** Ensures that at least one receiver acknowledges a transmitted frame.

2.1.6.1 Bit error

A bit error is detected when the monitored bit differs from the transmitted bit.

Exceptions:

- A DB observed during arbitration or in the ACK slot when a Recessive Bit (RB, logical '1') was sent (excluding dynamic stuff bits).
- A node sending a PEF and detecting a DB.

2.1.6.2 Stuff error

A stuff error is detected when a sixth consecutive identical bit appears in a field that follows dynamic bit stuffing rules. This applies to the MAC frame fields from SOF up to and including the CRC.

2.1.6.3 CRC error

A CRC error is detected when the received CRC sequence does not match the calculated CRC based on the observed bits.

2.1.6.4 Form error

A form error occurs when a fixed-format bit field contains one or more unexpected bit values.

Exceptions:

- A receiver detecting a DB at the last bit of EOF.

- A node detecting a DB at the last bit of the Error Frame Delimiter (EFD) or Overload Frame Delimiter (OFD).

2.1.6.5 ACK error

A transmitter detects an ACK error if no DB is received during the ACK slot.

2.1.6.6 Overload

An overload condition is detected under the following circumstances:

- A DB is detected during the first two bits of intermissionj (INT).
- A DB is detected by a receiver at the last bit of EOF.
- A dominant bit is detected by any node at the last bit of the OFD or EFD.

Detection of an overload condition does not increment either TEC or REC.

2.1.7 The LLC Frame Structure

The LLC frame consists of 15 bytes, see Table 2. Some bytes do not contain a full byte of data and unused bits are set to '0'. The LLC frame, along with the CAN_Ctrl module control signals, defines the CAN_Ctrl user interface.

Table 2. LLC frame structure.

Byte(s)	Description
0:3	Standard ID frames (11-bit ID, IDE = DB): ID(10:8) in Byte(2)(2:0), ID(7:0) in Byte(3). Extended ID frames (29-bit ID, IDE = RB): ID(28:24) in Byte(0)(4:0), ID(23:0) in Byte(1:3).
4	4-bit Data Length Code (DLC): '0000_DLC(3:0)'
5:12	8-byte data payload. MSB in Byte(5) and LSB in Byte(12)
13	Identifier Extension (IDE) bit: '0000_000(IDE)'.
14	Remote Transmission Request (RTR) bit: '0000_000(RTR)'.

2.1.8 The MAC Frame Structure

The MAC frame is the format transmitted on the CAN bus. It is derived from an LLC frame (see Table 2) and converted into either a DF or RF for transmission. On reception, the corresponding LLC frame is reconstructed from the received MAC frame. The CAN 2.0A MAC frame (11-bit ID) is detailed in Table 3, while the CAN 2.0B MAC frame (29-bit ID) is described in Table 4. The MAC frame bits are transmitted in the order they appear in MAC frame tables.

Table 3. CAN 2.0A MAC frame structure. The frame bits are transmitted in the order they appear in the table.

Frame structure			Bit Value	Description
SOF			DB	Marks the beginning of frame transmission.
Arbitration field	Base ID	Bit(10)	(D/R)B	Base ID from the corresponding LLC frame.
		...	(D/R)B	
		Bit(0)	(D/R)B	
	RTR		(D/R)B	DB = data frame (DF) =, RB = remote frame (RF).
Control field	IDE		DB	The DB value identifies the frame as CAN 2.0A.
	r0		DB	Reserved bit. Must be transmitted DB but accepted as either DB or RB by receivers.
	DCL	Bit(3)	(D/R)B	DLC from the corresponding LLC frame. If the frame is a RF, the DLC specifies the expected number of data bytes in the requested frame.
		...	(D/R)B	
		Bit(0)	(D/R)B	
Data field	Byte 0	Bit(7)	(D/R)B	Data bytes from the corresponding LLC frame. The field contains the number of bytes specified by the DLC. This field is omitted if the frame is a RF.
		..	(D/R)B	
		Bit(0)	(D/R)B	
	...		(D/R)B	
	Byte DLC	Bit(7)	(D/R)B	
		...	(D/R)B	
		Bit(0)	(D/R)B	
CRC field	CRC 15	Bit(14)	(D/R)B	15 bit CRC value.
		...	(D/R)B	
		Bit(0)	(D/R)B	
	CRC Delimiter		RB	Marks the end of the CRC field.
ACK	ACK Slot		(D/R)B	Transmitter sends RB, a receiver asserts DB if CRC is valid.
	ACK Delimiter		RB	Marks the end of the ACK field.
EOF	EOF(0)		RB	Marks the end of a valid CAN MAC frame.
	...		RB	
	EOF(6)		RB	

Table 4. CAN 2.0B MAC frame structure. The frame bits are transmitted in the order they appear in the table.

Frame structure			Bit value	Description
SOF			DB	Marks the beginning of frame transmission.
Arbitration field	Ext. ID	Bit(28)	(D/R)B	Extended ID from the corresponding LLC frame.
		...	(D/R)B	
		Bit(18)	(D/R)B	
	SRR		RB	Substitute Remote Request bit.
	IDE		RB	The RB value identifies the frame as CAN 2.0B.
	Ext. ID	Bit(17)	(D/R)B	Extended ID from the corresponding LLC frame.
		...	(D/R)B	
		Bit(0)	(D/R)B	
	RTR		(D/R)B	DB = data frame (DF) =, RB = remote frame (RF).
	Control field	r1	DB	Reserved bits. Must be transmitted DB but accepted as either DB or RB by receivers.
		r0	DB	
		DCL	Bit(3)	DLC from the corresponding LLC frame. If the frame is a RF, the DLC specifies the expected number of data bytes in the requested frame.
			...	
			Bit(0)	
Data field	Byte 0	Bit(7)	(D/R)B	Data bytes from the corresponding LLC frame. The field contains the number of bytes specified by the DLC. This field is omitted if the frame is a RF.
		..	(D/R)B	
		Bit(0)	(D/R)B	
	...		(D/R)B	
	Byte DLC	Bit(7)	(D/R)B	
		...	(D/R)B	
		Bit(0)	(D/R)B	
CRC field	CRC 15	Bit(14)	(D/R)B	15 bit CRC value.
		...	(D/R)B	
		Bit(0)	(D/R)B	
	CRC Delimiter		RB	Marks the end of the CRC field.
ACK	ACK Slot		(D/R)B	Transmitter sends RB, a receiver asserts DB if CRC is valid.
	ACK Delimiter		RB	Marks the end of the ACK field.
EOF	EOF(0)		RB	Marks the end of a valid CAN MAC frame.
	...		RB	
	EOF(6)		RB	

3. Requirements

The CAN_Ctrl node shall meet the following requirements:

1. Support CAN 2.0 A and B LLC frame formats (see Table 2).
2. The ns (node status) signal shall always indicate the current node state: '00' = EA, '01' = EP, '11' = BO (see section 2.1.4). '10' is unused.
3. When rst_i is asserted, all internal sub-modules shall reset, TEC and REC shall reset to 0, and the node shall enter the IDEL_S state (see Figure 5 and Table 6).
4. When ack_o is asserted, the last LLC frame received via the Avalon-ST input interface was successfully transmitted and acknowledged.
5. The node shall attempt re-transmission until an acknowledgment is received or the node enters the Bus-Off state.
6. Received LLC frames shall be streamed to the user via the Avalon-ST output interface.

4. CAN_Ctrl Module Design

The CAN_Ctrl module consists of six sub-modules:

1. A controlling Finite State Machine (can_fsm) (see section **Error! Reference source not found.**).
2. A Serial-To-Avalon-ST (STA) bridge converting the RX signal to a byte-wide Avalon-ST output stream (see section 4.1.2).
3. An Avalon-ST-To-Serial (ATS) bridge converting the Avalon-ST input bytes to a bit stream (see section 4.1.3).
4. A CRC module (CRCG) computing the 15-bit CRC sequence (see section 3.1.3).
5. A stuff bit generation module (SBG) which handles bit stuffing and de-stuffing by signaling the FSM when a valid stuff bit is ready.
6. A Node Clock (NC) module responsible for synchronization and generation of the RX sample pulse (s_rx). (see section 3.1.5).

The internal structure of CAN_Ctrl is illustrated in Figure 3, with Table 2 detailing the CAN_Ctrl module's input and output signals.

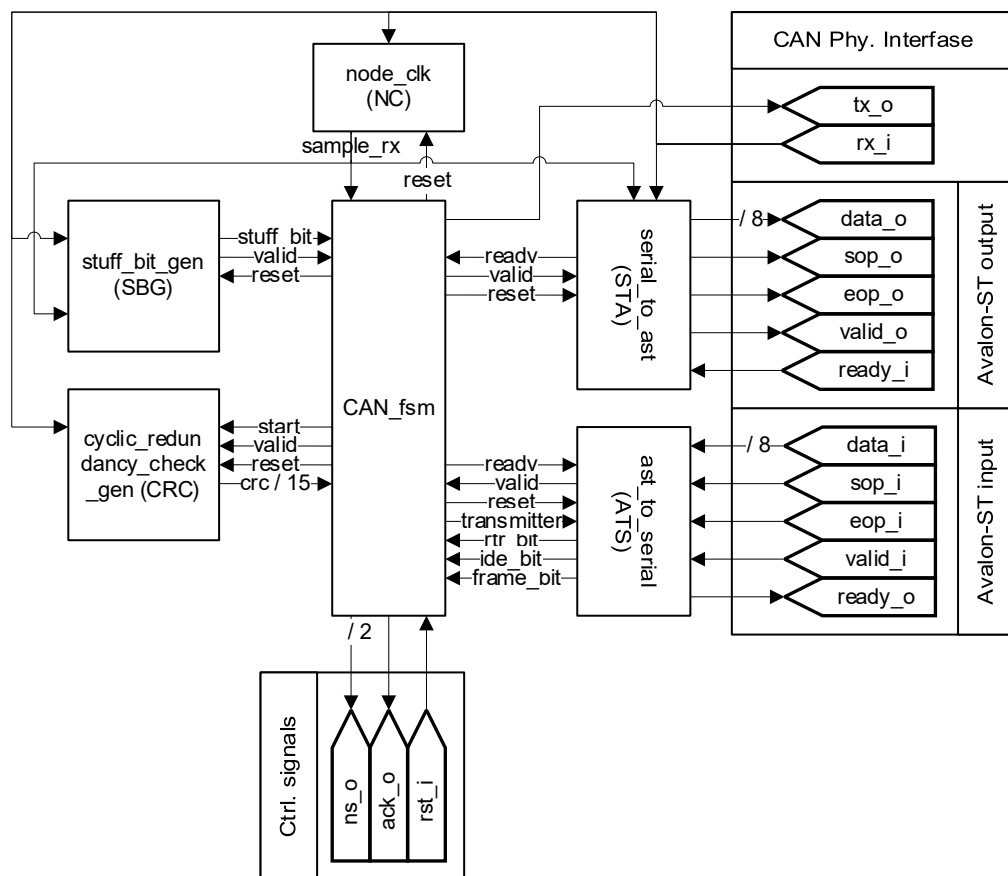


Figure 4. Internal structure of the CAN_Ctrl module. All signals are 1-bit unless otherwise specified.

Table 5. CAN_Ctrl input/output signals.

Name	Size	Source	Sink	Requirements	Guaranties
data_o	8 bits	CAN_Ctrl	User	--	Contain the next byte of a received LLC frame.
sop_o	1 bit	CAN_Ctrl	User	--	Asserted, when data_o contains the first byte of a received LLC frame.
eop_o	1 bit	CAN_Ctrl	User	--	Asserted, when data_o contains the last byte of a received valid LLC frame.
valid_o	1 bit	CAN_Ctrl	User	--	Asserted while data_o contains valid data.
ready_i	1 bit	User	CAN_Ctrl	Must be asserted for 1 system clock cycle.	On the rising edge of ready_i, the read pointer of the FIFO in C2A is advanced.
data_i	8 bits	User	CAN_Ctrl	Must be asserted for 1 system clock cycle. Must contain valid LLC frame bytes.	--
sop_i	1 bit	User	CAN_Ctrl	Must be asserted for 1 system clock cycle. Must be asserted when the first LLC frame byte is on data_i.	--
eop_i	1 bit	User	CAN_Ctrl	Must be asserted for 1 system clock cycle. Must be asserted when the last LLC frame byte is on data_i.	--.
valid_i	1 bit	User	CAN_Ctrl	Must be asserted for 1 system clock cycle. When asserted, the data on data_i, sop_i and eop_i must be valid.	--
ready_o	1 bit	CAN_Ctrl	User	--	when the ready_o signal is asserted, the data on data_i has transferred.
rst_i	1 bit	User	CAN_Ctrl	Must be asserted for 1 system clock cycle.	When rst_i is asserted, all internal sub-modules shall reset, TEC and REC shall reset to 0, and the node shall enter the IDEL_S state.
ack_o	1 bit	CAN_Ctrl	User	--	When asserted, the last LLC frame received via the Avalon-ST input interface was successfully transmitted and acknowledged
ns_o	2 bits	CAN_Ctrl	User	--	Encodes node status: '00' = EA, '01' = EP, '11' = BO. Value '10' is unused.
rx	1 bit	Phy	CAN_Ctrl	Must be connected to the physical CAN transceiver's RX line.	Receives bus state from the CAN network.

Name	Size	Source	Sink	Requirements	Guaranties
tx	1 bit	CAN_Ctrl	Phy	Must be connected to the physical CAN transceiver's TX line.	Outputs CAN frame data onto the physical bus.

4.1.1 CAN_fsm

The FSM controlling the CAN_Ctrl module (can_fsm) is depicted in Figure 5 and the states along with brief descriptions are tabulated in Table 6.

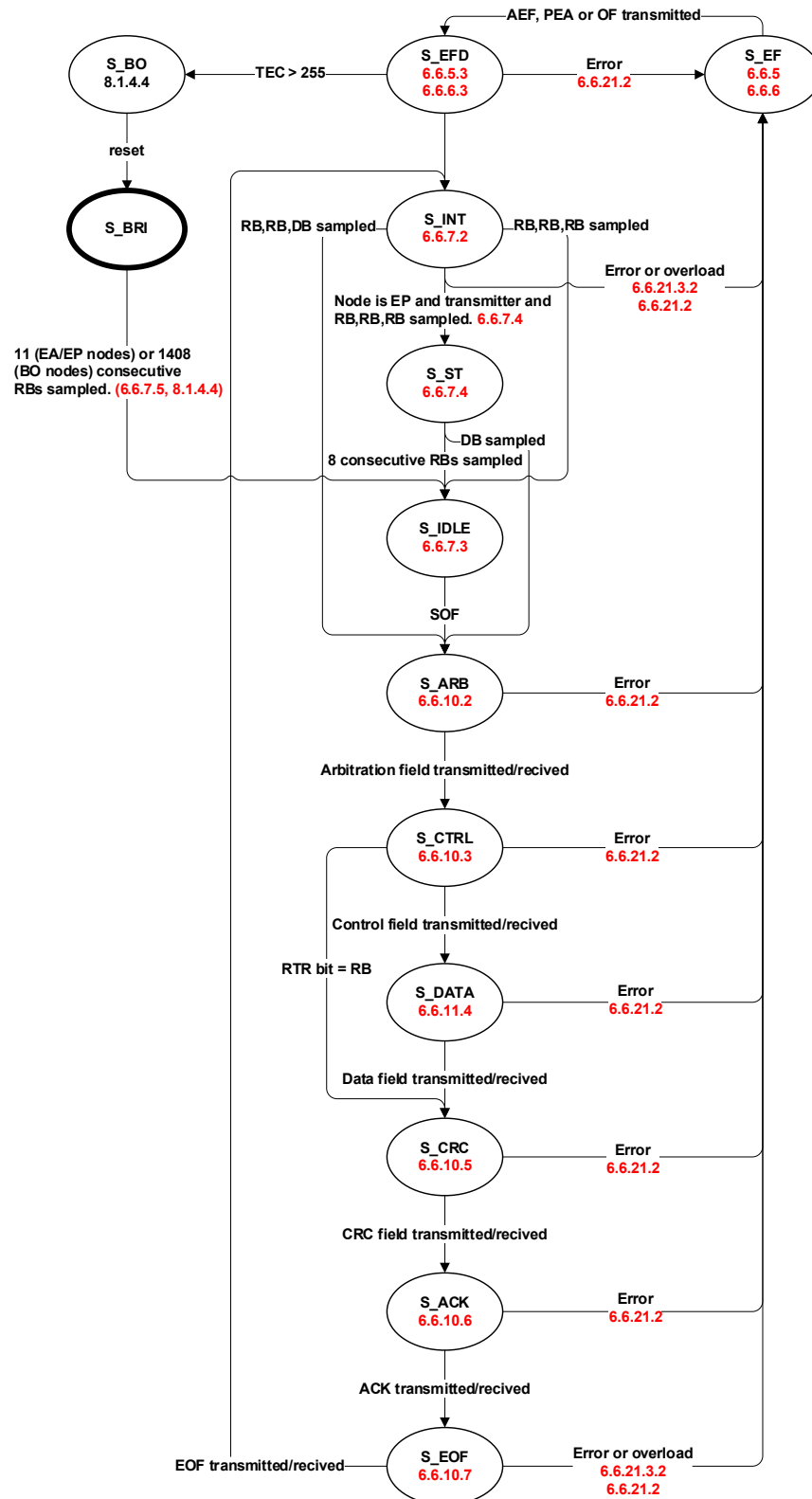


Figure 5. FSM controlling the CAN_Ctrl module (can_fsm). Red text are references to the relevant sections in the ISO standard. [1]

Table 6. States comprising the can_fsm controlling CAN_Ctrl.

Name	Description
s_bus_off	See section 2.1.4.3
s_bus_reintegration	The initial state of the CAN_Ctrl node. The node remains in this state until 11 consecutive RBs are detected on the bus.

Name	Description
s_error_frame	The node transmits an error frame (AEF / PEF / OF) corresponding to its error status (EA or EP).
s_error_frame_delimiter	The node transmits the EFD to mark the end of an error frame.
s_intermission	The node transmits the INT field. If a DB is observed in the third bit, it is interpreted as SOF. A dominant bit in the first or second bit is an overload condition, triggering an OF.
s_suspend_transmission	The node suspends transmission for eight BTs. If a DB is observed during this period, it is interpreted as SOF, and the node becomes a receiver. Only EP transmitters enter this state.
s_idle	The node enters this state when the bus is idle. Receivers and EA transmitters transition to IDLE_S when the third bit of INT field is RB. EP transmitters enter this state after the bus remains RB for eight BTs.
s_arbitration	The frame arbitration process occurs in this state, see section 2.1.2
s_control	The MAC frame control field is transmitted or received in this state.
s_data	The data payload is transmitted or received in this state.
s_crc	The transmitter sends its computed CRC value, while receivers compare it to their own calculated CRC to check for transmission errors.
s_ack	The transmitter sends a RB, and receivers with matching CRC values acknowledge reception by overwriting it with a DB. If the transmitter does not detect a DB in the ACK slot, it responds with an EF. Receivers with mismatching CRC values also respond with an EF.
s_end_of_frame	The EOF field is transmitted or received. If a DB is observed in the last bit of EOF, it is treated as an overload condition, triggering an OF.

Table 7. Helper states, not shown in Figure 5

Name	Description
s_error_frame_dominant_delimiter	Used to determine when there are no more dominant bits in an error flag. If there are too many it will count 8.1.4.2.f
s_to_error_frame	Used to set common values before going to the s_error_frame
s_to_arbitration	Used to set values before going to arbitration
s_error_frame_check	Used to check if there a one more dominant bit so we know that it as a receiver was the first one to report the error. 8.1.4.2.b
s_error_passive_wait	For error passive node to simply count for 6 equal bits. See 6.6.5.2, last paragraph.

4.1.2 Serial_to_Aavalon-ST (STA)

The internal structure of the STA sub-module is illustrated in Figure 6. It consists of a controlling FSM (sta_fsm), an 8-bit shift register, and memory with capacity for two LLC frames. When the valid signal from can_fsm to sta_fsm is asserted, the bit on RX is shifted into the shift register. When the appropriate number of bits have been collected, the resulting byte its written to memory. Assertion of the error signal tags the last LLC frame written to memory as invalid and it will be overwritten by the subsequent LLC frame. The overload signal will be asserted when the memory contains two valid LLC frames, prompting the

can_fsm to transmit OFs until at least one LLC frame has been read through the Avalon-ST interface.

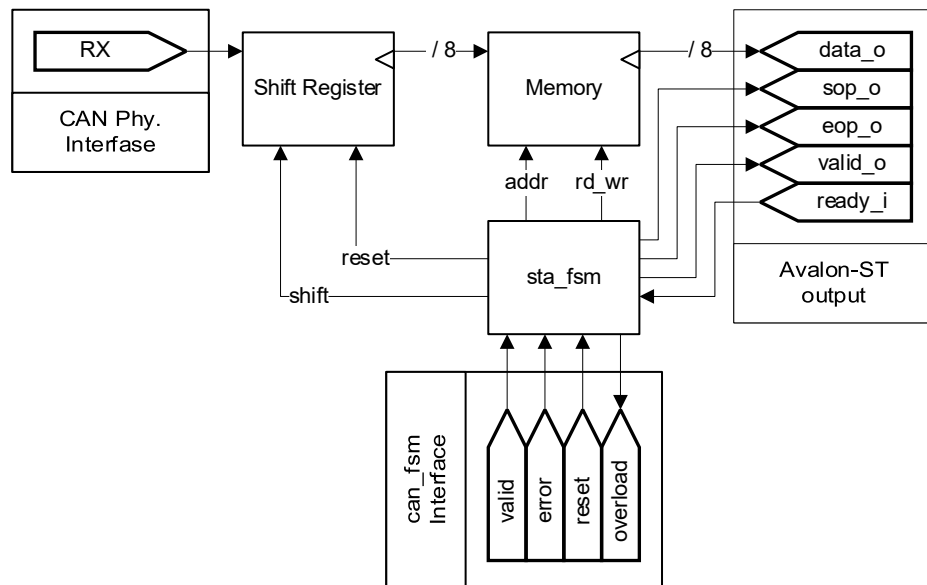


Figure 6. Structure of the Serial_to_Avalon-ST (STA) sub-module. Unless otherwise specified all signals are 1-bit.

4.1.3 Avalon-ST_to_Serial (ATS)

The internal structure of the ATS sub-module is illustrated in Figure 7. It consists of a controlling FSM (ats_fsm), an 8-bit shift register, and a 13-byte memory. When the ready signal from can_fsm to ats_fsm is asserted, the next bit of a stored the LLC frame is driven unto the frame_bit signal. The valid signal from ats_fsm to can_fsm indicates that valid data is present on the frame_bit signal. The shift register will be reset to output the MS bit of the frame ID, If the transmitter signal from can_fsm to ats_fsm is de-asserted before a full frame has been shifted out of memory.

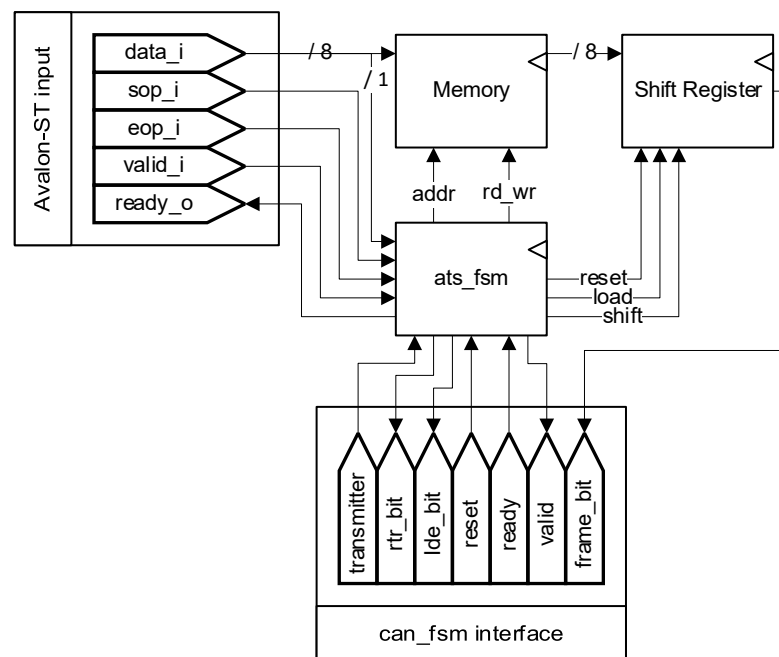


Figure 7. Structure of the Avalon-ST_to_Serial (ATS) sub-module. Unless otherwise specified all signals are 1-bit.

4.1.4 Node Clock (NC)

The NC is responsible for synchronization and generating the RX sample pulse (s_{rx}). It performs hard synchronization at SOF and resynchronizes on every subsequent RB-to-DB transition. The FSM controlling the CNC is illustrated in Figure 8.

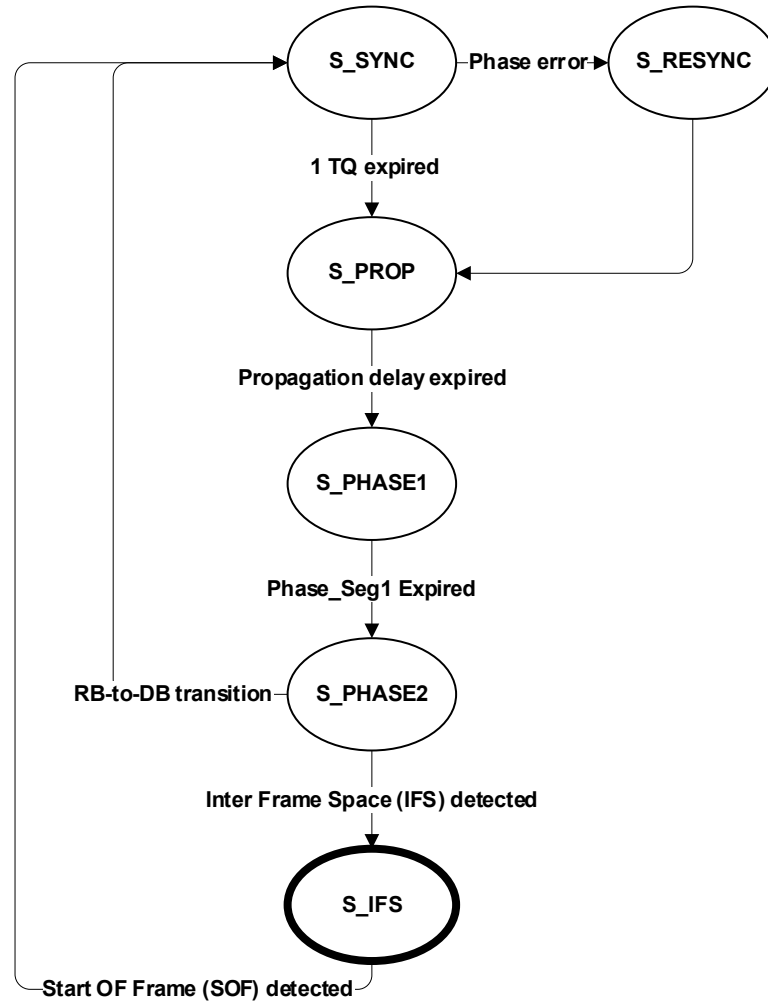


Figure 8. FSM controlling the NC.

4.1.5 CRC module

The CRC module shall compute the 15-bit CRC sequence using the following algorithm outlined in ISO 11898-1:2024 standard. The used CRC polynomial is CRC_POLYNOMIAL = 0xC599 in hexadecimal notation.

```

CRC_RG(nCRC -1: 0) = CRC_INIT_VECTOR; // Initialize shift register
REPEAT
    CRCNXT = NXTBIT EXOR CRC_RG(nCRC -1);
    CRC_RG(nCRC -1: 1) = CRC_RG(nCRC -2: 0); // Shift left by one position
    CRC_RG(0) = 0;
    IF CRCNXT THEN
        CRC_RG(nCRC -1: 0) = CRC_RG(nCRC -1: 0) EXOR (CRC_POLYNOMIAL);
    ENDIF
UNTIL (NXTBIT = [End of bit stream] OR an error condition occurs).
    
```